

QA Sprint 1

Telemetry Contract, Decision Log and Open Questions

Purpose: Establish an unambiguous, testable contract for telemetry identity, timestamps, partial batch outcomes and live alert ordering before the firmware, gateway and backend implementations diverge.

Document field	Value
Project	Industrial Equipment Monitoring Platform
Role perspective	Quality Assurance (QA)
Sprint / milestone	Week 1 - M1: Contracts and vertical slice
Requirements baseline	Version 1.0 - Draft baseline
Primary requirement references	FR-05, FR-08, FR-09, FR-11, FR-14; AT-05, AT-06; M1
Status	Draft for team review and decision confirmation

Source basis

This document is derived from the uploaded Industrial Equipment Monitoring Platform Requirements Specification v1.0. It preserves the baseline requirements and explicitly labels proposed QA interpretations and unresolved questions rather than treating them as approved requirements.

1. Sprint Goal and QA Scope

Sprint Goal: Establish a testable, versioned telemetry contract and prove that simulated device events can travel through the ingestion API into persistent storage with deterministic identity, timestamp, validation, duplicate and partial-batch behaviour.

The requirements define M1 as “Contracts and vertical slice” and target requirements, telemetry schema, database migrations and simulator-to-API ingestion for Week 1. The physical ESP32-through-gateway dashboard demonstration is a Week 2 milestone, so Sprint 1 QA should avoid expanding into the full physical path prematurely.

1.1 In scope for Sprint 1 QA

- Telemetry schema validation and required-field behaviour.
- Stable event identity and database uniqueness semantics.
- Idempotent duplicate handling and conflicting-identity handling.
- Timestamp meaning, trust and storage expectations.
- Per-item outcomes for mixed telemetry batches.

- Database-commit semantics before an event is reported as accepted.
- Simulator-to-API integration and persistence evidence.
- Alert ordering rules clarified sufficiently to avoid incompatible data-model decisions.

1.2 Out of scope for Sprint 1 execution

- Full ESP32 -> MQTT -> Raspberry Pi -> backend -> dashboard physical verification (M2).
- Complete durable buffering/retry/restart reliability campaign (M3), although the contract must support it.
- Full authentication/site-permission and operator alert workflow acceptance (M4).
- Performance, security release verification and backup/restore evidence planned for later milestones.

2. Event Identity and Idempotency

The baseline requires each reading to have a stable identity. Retries retain the same device ID, boot ID and sequence number; a new boot receives a new boot ID. The database must enforce uniqueness on the same tuple.

QA-D01 - Canonical event identity: Use (device_id, boot_id, sequence_number) as the authoritative identity for a telemetry event. Timestamps and payload hashes are not identity fields.

```
event_identity = (device_id, boot_id, sequence_number)
```

Example:

device_id = pump-01

boot_id = 550e8400-e29b-41d4-a716-446655440000

sequence_number = 152

Case	Example	Expected QA result
Retry of same reading	pump-01 / boot-A / 152	Same event identity; no second stored telemetry row.
Next reading	pump-01 / boot-A / 153	New event.
Device reboot	pump-01 / boot-B / 1	New event because boot_id changed.
Identity reused with altered content	pump-01 / boot-A / 152, but value differs	Reject and log an identity/content conflict; do not update the original event.

2.1 Minimum test conditions

- Submit an identical event ten times and verify exactly one stored row.
- Submit the same identity with a different temperature and verify explicit rejection and logging.
- Restart the simulated device, generate a new boot_id and verify sequence_number may restart without collision.
- Verify the database has a unique constraint/index on (device_id, boot_id, sequence_number), not only application-side checks.

3. Timestamp Semantics

The telemetry contract separates device measurement time from gateway and backend receipt time. This distinction is required for historical display, freshness measurement, replay handling and debugging.

Field	Meaning	Assigned by	Primary QA use
measured_at	UTC time when the physical measurement occurred; null when device time is not synchronised.	Device	Historical timeline when clock is trustworthy.
device_uptime_ms	Elapsed time since the current device boot.	Device	Ordering/debugging within a boot and clock fallback context.
gateway_received_at	UTC time the gateway receives the reading.	Gateway	Freshness and gateway-side pipeline timing.
backend_received_at	UTC time the backend receives the event.	Backend	Ingestion audit, replay-age decisions and server-side latency.

QA-D02 - Timestamp storage: Store the distinct time concepts independently. Never silently replace an unknown `measured_at` with `gateway_received_at` or `backend_received_at`. All stored timestamps remain UTC; display timezone is a presentation concern.

```
{
  "measured_at": null,
  "device_uptime_ms": 83421,
  "gateway_received_at": "2026-09-18T03:22:01.321Z",
  "backend_received_at": "2026-09-18T03:22:02.017Z"
}
```

3.1 Open timestamp questions

ID	Question	Why QA needs it resolved
QA-OQ01	What exact values are supported for clock/reading quality?	The baseline requires explicit quality information but does not define an enum or validation rules.
QA-OQ03	Which timestamps determine whether a reading is more than 30 seconds old for live alert evaluation?	Replay behaviour must be deterministic when <code>measured_at</code> is null or untrusted.
QA-OQ04	Which timeline is used for the >15-second consecutive-reading gap rule?	A rule based on <code>measured_at</code> behaves differently from uptime or receive-time when clocks are unsynchronised.

A possible quality vocabulary for team discussion is SYNCED / UNSYNCED / ESTIMATED / UNKNOWN. This is a proposal only; it is not defined by the requirements baseline.

4. Partial Batch Outcomes and Queue Behaviour

FR-09 requires validation per batch item: valid items in a mixed batch can still succeed. Processing rules further require each response item to be identified as accepted, duplicate or rejected, with gateway queue handling based on the individual outcome.

QA-D03 - Batch atomicity at the application level: Do not treat a mixed telemetry batch as all-or-nothing. Return a deterministic outcome for every item so the gateway can remove, quarantine or retry each event correctly.

Batch

event 101 -> valid
 event 102 -> duplicate
 event 103 -> invalid unit
 event 104 -> valid

Expected per-item outcome

101 -> accepted
 102 -> duplicate
 103 -> rejected: INVALID_UNIT
 104 -> accepted

Outcome	Backend meaning	Gateway action
accepted	Database transaction committed successfully.	Remove event from queue.
duplicate	Matching event identity/content already stored.	Remove event from queue.
rejected - permanent	Schema/ownership/unit/value/etc. is permanently invalid.	Move to bounded quarantine and record rejection reason.
transient failure	Backend unavailable, timeout, rate limit or server failure.	Keep queued and retry using bounded backoff.

4.1 Lost response after commit

A critical failure mode is: backend commits the batch, but the network response is lost. The gateway cannot know that the commit succeeded and therefore retries. Idempotency must convert the retry into duplicate outcomes without creating new rows. This behaviour is directly exercised by acceptance scenario AT-05.

```

POST batch
|
backend commits
|
response is interrupted
|
gateway retries same identities
|
backend returns duplicate for already-committed events
|
database row counts remain unchanged
  
```

4.2 Open batch questions

- QA-OQ05 - Define exactly which persisted fields are compared when distinguishing a matching duplicate from an identity/content conflict.

- QA-OQ06 - Define HTTP status semantics for a syntactically valid batch containing mixed accepted/duplicate/rejected outcomes.
- QA-OQ07 - Define stable machine-readable rejection codes so gateway quarantine logic does not depend on free-form text.

5. Alert Ordering, Stale Events and Replay

The baseline states that three consecutive valid readings above the configured threshold create one active alert; a gap longer than 15 seconds resets a pending count; invalid readings do not count; readings more than 30 seconds old when received by the backend are historical only; and duplicate or out-of-order readings do not advance live evaluation.

QA-D04 - Proposed within-boot ordering: For a given device_id + boot_id, use sequence_number to decide live progression. A reading that arrives with a lower sequence number than the highest live-evaluated sequence is stored historically but does not advance or rewind live alert state. This interpretation requires team approval.

sequence values: 100, 101, 102, 103
arrival order: 100 -> 102 -> 103 -> 101

Proposed result:

- 100 / 102 / 103 can advance live state subject to consecutive/gap rules.
- delayed 101 is retained in history but cannot rewrite already-advanced live state.

5.1 Cross-boot ordering

Sequence numbers are only monotonic within a boot, so sequence_number alone cannot compare boot-A/500 with boot-B/1. The baseline does not fully define how a new boot supersedes an older boot in live state.

QA-OQ02 - Cross-boot ordering: Define how the backend decides that a newly observed boot becomes the current live stream, and ensure delayed telemetry from an older boot can populate history without replacing current live state.

10:00 boot-A / seq 500
10:01 device reboots
10:01 boot-B / seq 1
10:02 boot-B / seq 2
10:04 delayed boot-A / seq 499 arrives

Required property:

old boot-A data must not become the device's latest live temperature after boot-B has established the current stream.

5.2 Replay behaviour

Historical replay and live state must be separated. Old readings may be stored in history, but stale, duplicate and out-of-order readings must not advance live temperature-alert evaluation. Similarly, replayed historical telemetry must not make a physically stopped device appear online.

```

replayed backlog
|
+--> history persistence
|
+--X live alert advancement when stale/out-of-order/duplicate
|
+--X connection "online" state based only on historical replay

```

6. QA Decision and Open-Question Register

ID	Topic	Status	Decision / question
QA-D01	Event identity	Decision	(device_id, boot_id, sequence_number) is the canonical identity.
QA-D02	Timestamp storage	Decision	Keep device, gateway and backend time concepts separately; store timestamps in UTC.
QA-D03	Batch processing	Decision	Return per-item accepted / duplicate / rejected outcomes; classify transient failures separately for retry.
QA-D04	Within-boot ordering	Proposed	Use sequence_number for live progression within a boot; out-of-order events remain historical only.
QA-OQ01	Clock quality	Open	Define supported quality values and validation semantics.
QA-OQ02	Cross-boot ordering	Open	Define how a new boot supersedes an old boot for live state.
QA-OQ03	Stale-event calculation	Open	Define the timestamps/rules used for the 30-second live-alert age test.
QA-OQ04	Gap detection	Open	Define the timeline used for the >15-second consecutive-reading gap.
QA-OQ05	Duplicate comparison	Open	Define matching duplicate versus identity/content conflict comparison fields.
QA-OQ06	Batch HTTP semantics	Open	Define HTTP status behaviour for mixed per-item outcomes.
QA-OQ07	Rejection taxonomy	Open	Define stable rejection codes and permanence/transience classification.

6.1 Suggested owners for clarification

Topic	Primary owner(s)	QA expectation
Identity + duplicate conflict	Backend Lead + Gateway Developer	Publish contract examples and automated tests.
Clock quality + timestamp semantics	Firmware Developer + Gateway Developer + Backend Lead	Agree enum/nullable rules and canonical age/gap calculations.
Cross-boot ordering	Backend Lead / Architect	Document live-state transition logic and edge cases.
Batch response + rejection codes	Backend Lead + Gateway Developer	Version API response schema and map outcomes to queue actions.
Alert ordering	Backend Lead + QA	Write executable test matrix before alert implementation begins.

7. Sprint 1 QA Verification Matrix

Test ID	Scenario	Expected result	Req. link
S1-QA-01	Submit valid simulator event.	Event persists once with all required identity and timestamp fields validated.	FR-05, M1

Test ID	Scenario	Expected result	Req. link
S1-QA-02	Submit exact same event identity/content 10 times.	One stored row; retries are reported as duplicates.	FR-08
S1-QA-03	Reuse same identity with changed value/content.	Explicit rejection and log; original stored record unchanged.	FR-08
S1-QA-04	Submit mixed valid, duplicate and invalid items.	Each item receives correct outcome; valid events succeed independently.	FR-09, AT-06
S1-QA-05	Commit then interrupt response and retry.	Retry is safe; database counts remain correct.	AT-05
S1-QA-06	Send unsupported schema, invalid unit/value, unknown device.	Stable rejection code/reason for each invalid item.	FR-09
S1-QA-07	Send measured_at=null with unsynchronised clock quality.	Event accepted/rejected according to agreed quality rules without fabricated measurement time.	Telemetry contract
S1-QA-08	Send lower sequence after a higher sequence in same boot.	Stored historically; does not replace newer current/live state under approved ordering rule.	Alert processing
S1-QA-09	Start new boot_id and later replay old-boot event.	Old-boot replay cannot replace established new-boot live state.	Open clarification
S1-QA-10	Verify database uniqueness definition.	Constraint exists on (device_id, boot_id, sequence_number).	Telemetry rules

8. Sprint 1 Exit Criteria for QA Sign-off

QA should recommend M1 sign-off only when the telemetry contract is sufficiently deterministic that independent firmware, gateway, backend and QA implementations predict the same result for the same event.

- Versioned telemetry request/response schema is committed to the repository.
- Database migrations include the required event-identity uniqueness constraint.
- Simulator-to-API ingestion succeeds and persistence can be verified.
- Automated tests cover valid, duplicate, conflicting-identity and mixed-batch cases.
- Acceptance is never reported before the relevant database transaction commits.
- Machine-readable per-item response outcomes are defined and tested.
- Timestamp and quality semantics are documented; unresolved behaviour is explicitly tracked rather than assumed.
- Alert-ordering decisions required by the data model are approved or formally recorded as blocking open questions.
- No blocking defect remains for the M1 contract/vertical-slice scope.

QA sign-off standard: A firmware developer, gateway developer, backend developer and QA engineer should independently read the versioned contract and predict the same outcome for a duplicate, reordered, stale, partially invalid or replayed event.

9. Requirements Traceability Notes

Requirement area	Baseline statement used by this QA document
FR-05	Every reading has stable event identity; retries retain device ID, boot ID and sequence number; new boot gets new boot ID.

Requirement area	Baseline statement used by this QA document
FR-08	Ingestion is idempotent; same event repeated creates one stored record; same identity with different content is rejected and logged.
FR-09	Batch items are validated individually; valid items in a mixed batch can still succeed.
Telemetry processing	Uniqueness on (device_id, boot_id, sequence_number); per-item accepted/duplicate/rejected outcomes; queue removal/quarantine/retry rules.
Alert behaviour	Stale, duplicate and out-of-order readings do not advance live alert evaluation; >15-second gap resets pending count.
AT-05 / AT-06	Lost backend response after commit must retry safely; mixed valid/invalid/duplicate batch must return correct individual outcomes.
M1 / Definition of Done	Week 1 delivers contracts, schema, migrations and simulator-to-API ingestion; implementation/tests/docs must link to requirements and leave no blocking defect.

End of QA Sprint 1 Working Document